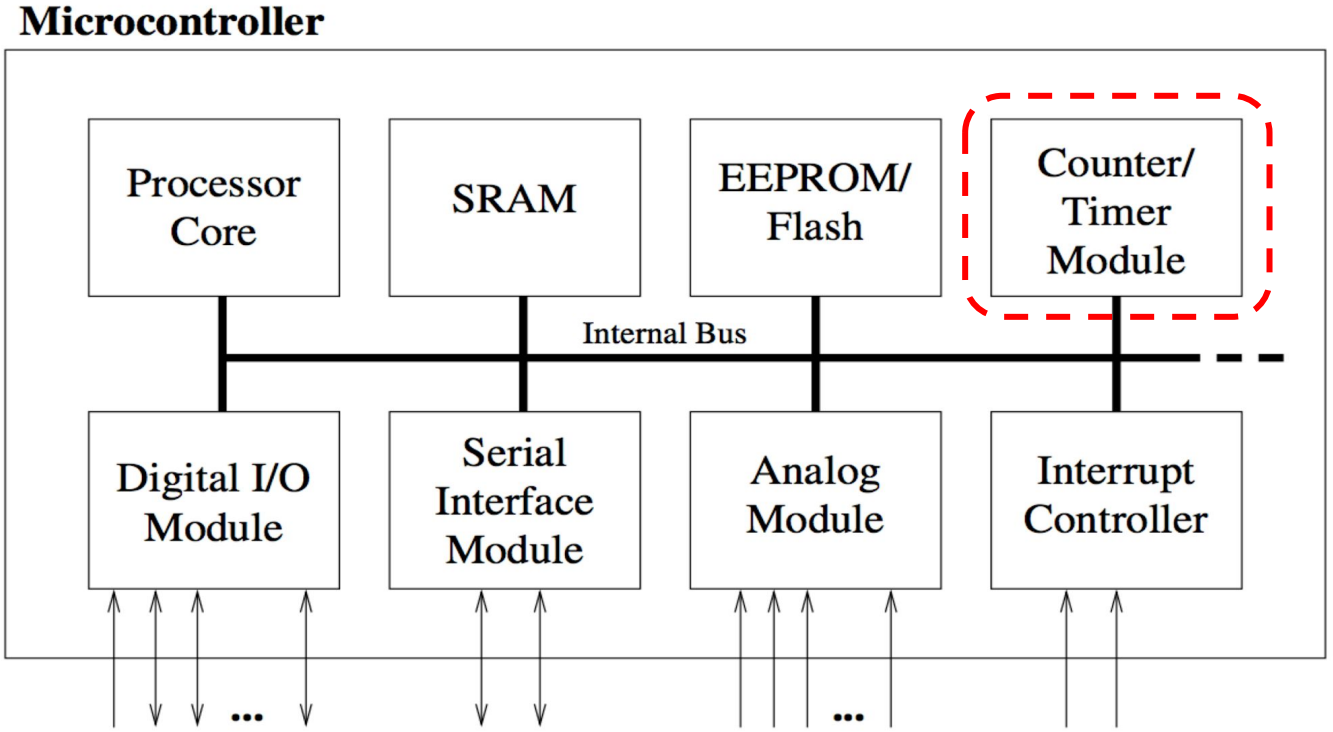
UNIVERSITÀ
degli STUDI
di CATANIA

IOT Systems and Technologies

Introduction to Microcontrollers: Timers

davide.patti@dieei.unict.it

Basic Layout of a Microcontroller



- What is a Hardware Timer? It is a dedicated digital counter built into the microcontroller's silicon.
- It operates independently of the main CPU, meaning it counts time without slowing down your code.

Microcontroller Timers: Core Functions

The most basic use of the timer is as a counter, but modern microcontrollers offer advanced functionality for complex control systems.

Counter

Standard counting of clock cycles or external events.

Input Capture

Used to timestamp external events with high precision.

Output Compare

Triggers interrupts or changes output pin state after a specific number of clock cycles.

PWM

Pulse Width Modulation signals generated for precise motor control and power regulation.

Timer Fundamentals: The Counter

Clock Tick Operation

Every timer is essentially a counter that increments or decrements on every clock tick. The direction can be fixed or software-configurable.

Register Access

The current count value is readable through a dedicated count register. Users can also initialize the counter by setting it to a specific value.

Timer Resolution and Range

For a timer resolution of **n** bits, the count value range is defined as: $[0, 2^n - 1]$

Sources for Clocking the Timer

System Clock

Internal clock source used for synchronized operation.

Prescaler

Divides the system clock to provide lower frequencies.

External Pulse

Increments the timer based on external signal events.

Source: System Clock

Internal Clock Logic

Often called "Internal Clock" because it is the primary clocking source for the entire controller.

- The oscillator itself can be external.

Timer Synchronization

The timer is incremented precisely with every tick of the system clock.

- Ensures synchronous operation across modules.

Note: The term "internal" exclusively highlights that this is the clocking source used by the entire controller architecture.

Prescaler Fundamentals

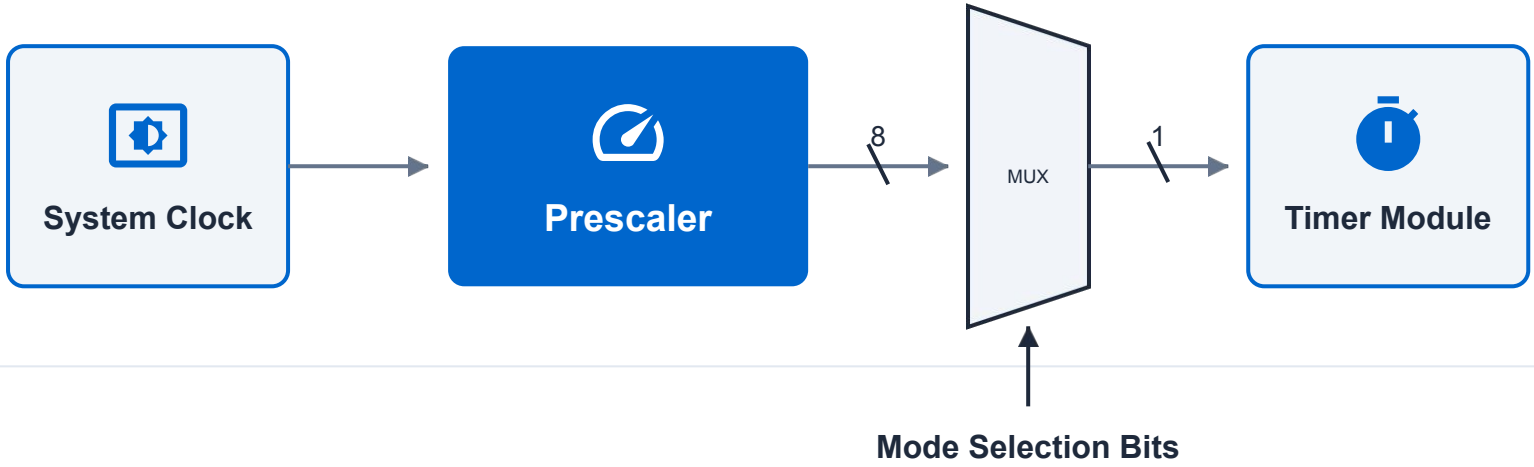
A prescaler is a variable-length counter (typically 8 or 10 bits) incremented by the system clock to divide frequency.

Practical Example: Bit Timing

The prescaler increments with every rising edge of the system clock.

The **least significant bit** of the prescaler will have **twice the period** of the system clock, effectively halving the frequency.

Prescaler Architecture & Selection



Frequency Division Mechanism

Each subsequent bit in the prescaler chain divides the frequency by two sequentially.

Configurable Prescale Factors

The timer module's mode bits enable selection of various factors (e.g., 8, 64, 256, 1024) to optimize range versus granularity.

Prescaler Trade-offs: Range vs. Granularity

Increasing the prescaler extends the timer's range but results in coarser granularity and larger measurement errors.

No Prescale (1:1)

8-bit timer @ 1 MHz

- Range: 0 – 255 µs
- Resolution: 1 µs

Prescale Value: 2

Double the range

- Range: 0 – 511 µs
- Resolution: 2 µs

Prescale Value: 1024

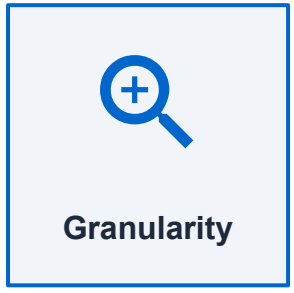
Maximum duration

- Range: 0 – ~260 ms
- Resolution: ~1 ms

Key Conclusion

While a prescaled timer can measure longer durations, it comes at the direct cost of measurement precision.

Prescaler Optimization



Core Principle: Smallest Value Selection

Use the smallest prescaler value that fits the application's specific requirements.

- Achieve the best possible granularity from available choices.

Design Tip

Higher resolution measurements allow for more precise control and timing accuracy within the system firmware.

External Pulse Clocking

External Signal Source

The timer derives its clock from an external signal connected to a specific controller input pin.

Triggering Mechanism:
Increments count on observed transitions, such as a rising edge.

Sampling Constraints

Since it's sampled like any other input, specific timing rules apply to ensure detection.

Minimum Pulse Width:
Time between edges must exceed one system clock cycle.

Design Note

Ensure the external frequency does not exceed the sampling capabilities of the internal system clock to avoid missed pulses.

Input Capture Fundamentals

Core Function: Event Timing

Input capture is primarily used to accurately timestamp occurrences within the system.

- Supports both **External** and **Internal** events.
- Triggers on **Edges** (rising/falling) or specific **Levels**.

Implementation Detail

Precision timing depends on the system clock frequency and the edge detection sensitivity of the chosen timer module.

Input Capture: Event Handling

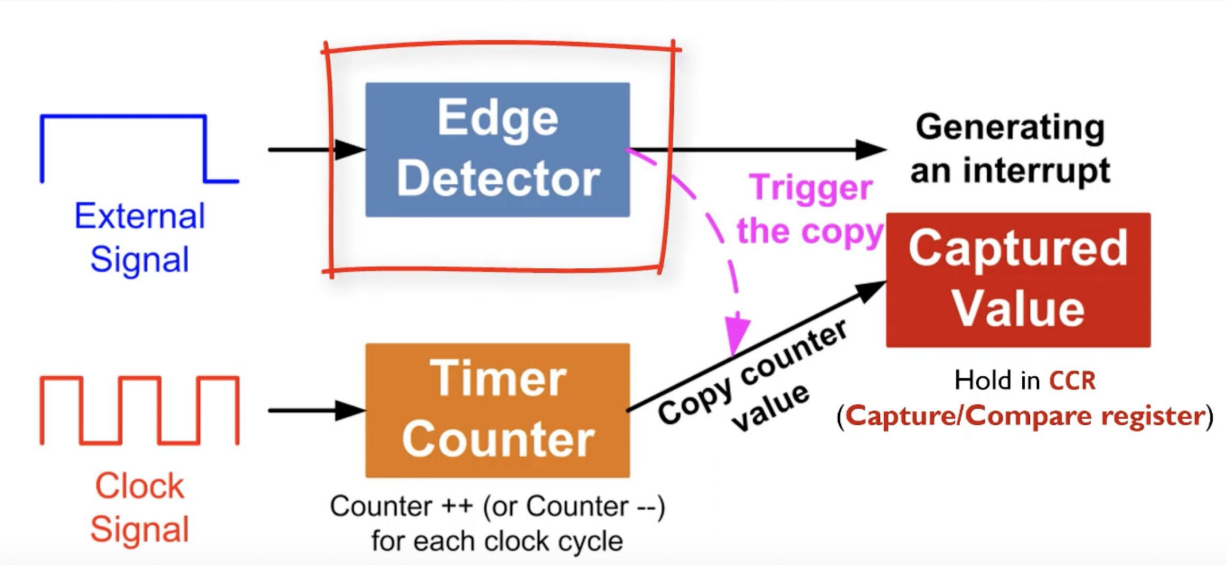
Automated Hardware Response

Whenever a pre-defined event occurs on the input pin:

- The timer **automatically copies** its current count to an input capture register.
- The **input capture flag** is set immediately.
- An **interrupt request** can be triggered for the CPU.

Key Advantage

The hardware-level copy operation ensures zero-latency timestamping, unaffected by software execution delays.



Output Compare Mechanisms

Input Capture

The system records a timestamp when a specific event occurs on the input line.

- Triggered by signal edges
- Stores counter value in register

Output Compare

The system triggers an action on the output line once a target time is reached.

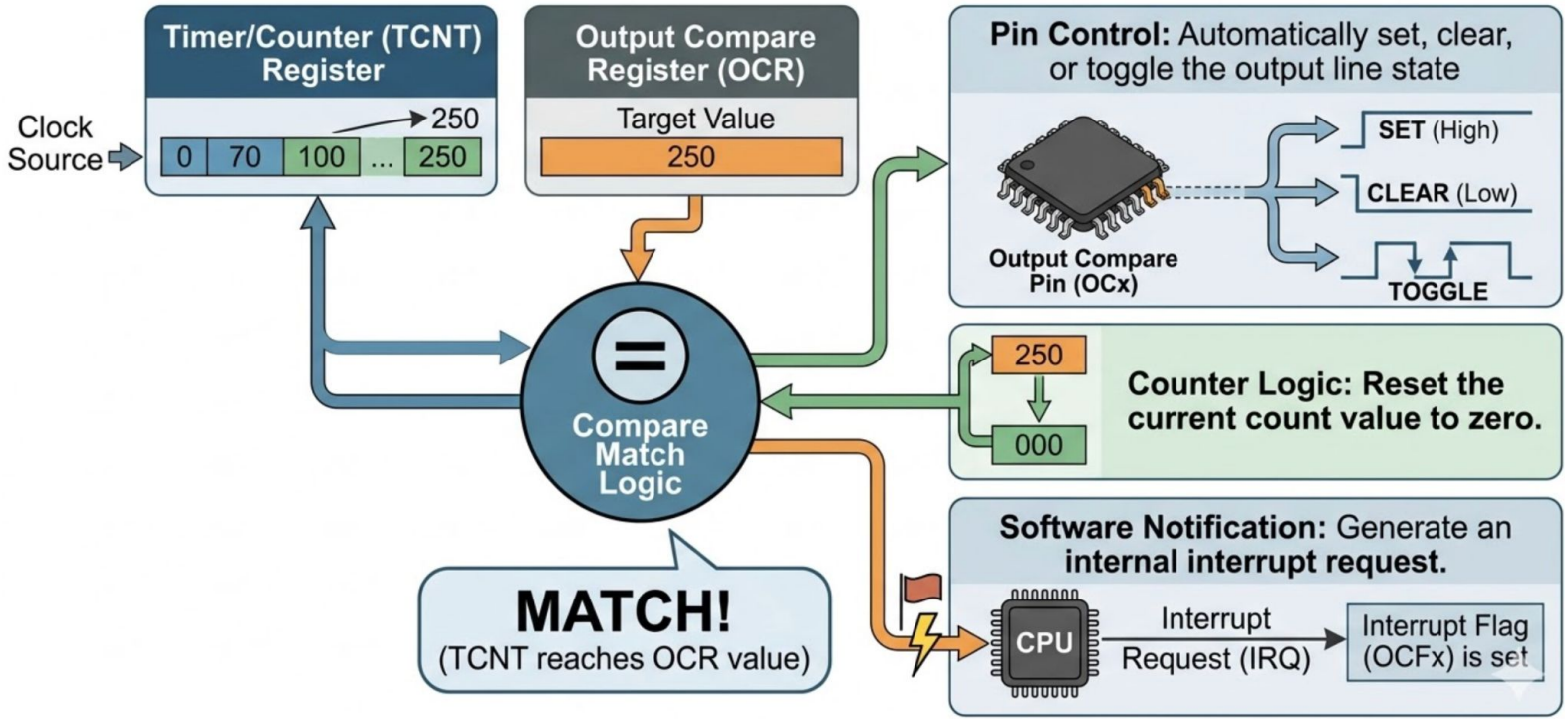
- Counter-driven events
- Counterpart of Input Capture

Functional Relationship
While Input Capture listens for events to record time, Output Compare uses time to trigger events.

Output Compare: Advanced Functions

The timer utilizes an **output compare register** to set target values. **When counter reaches compare value:**

- **Pin Control:** Automatically set, clear, or toggle the output line state.
- **Counter Logic:** Reset the current count value to zero.
- **Software Notification:** Generate an internal interrupt request.



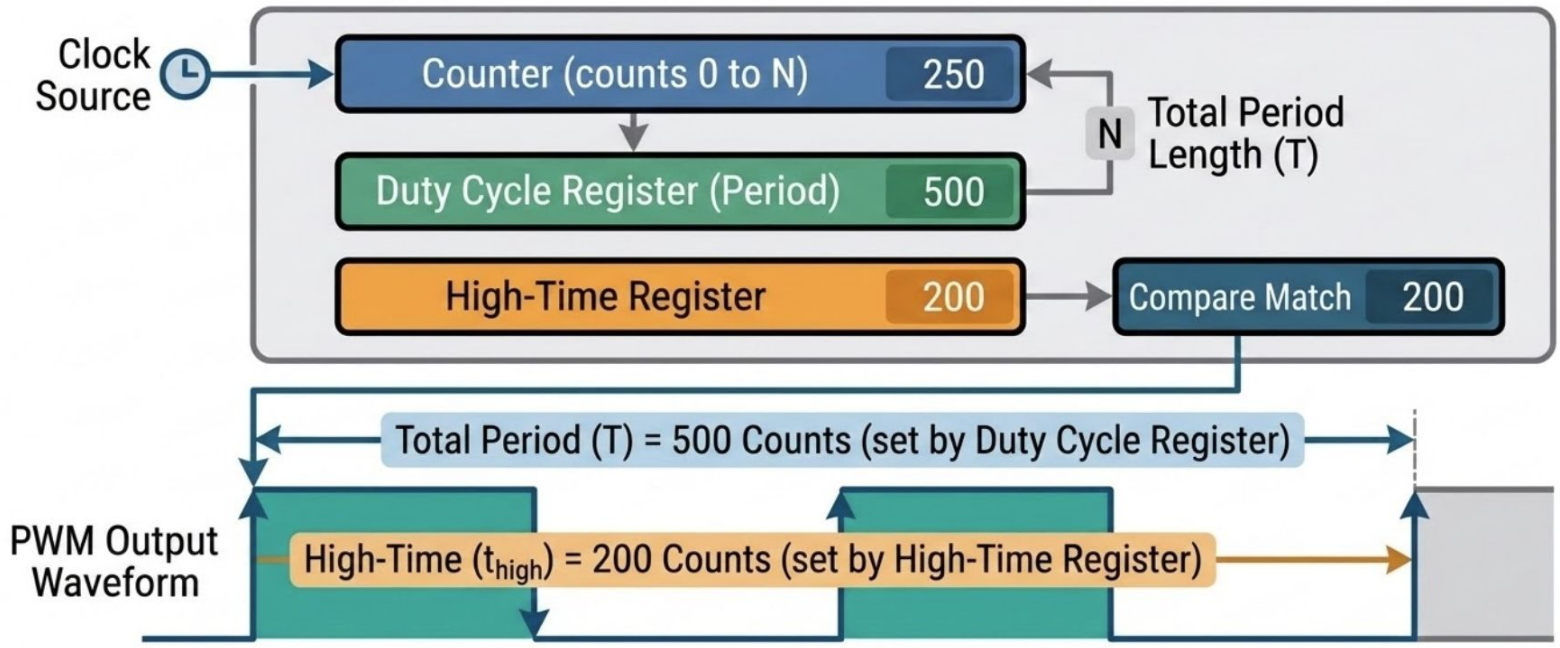
Pulse Width Modulation (PWM)

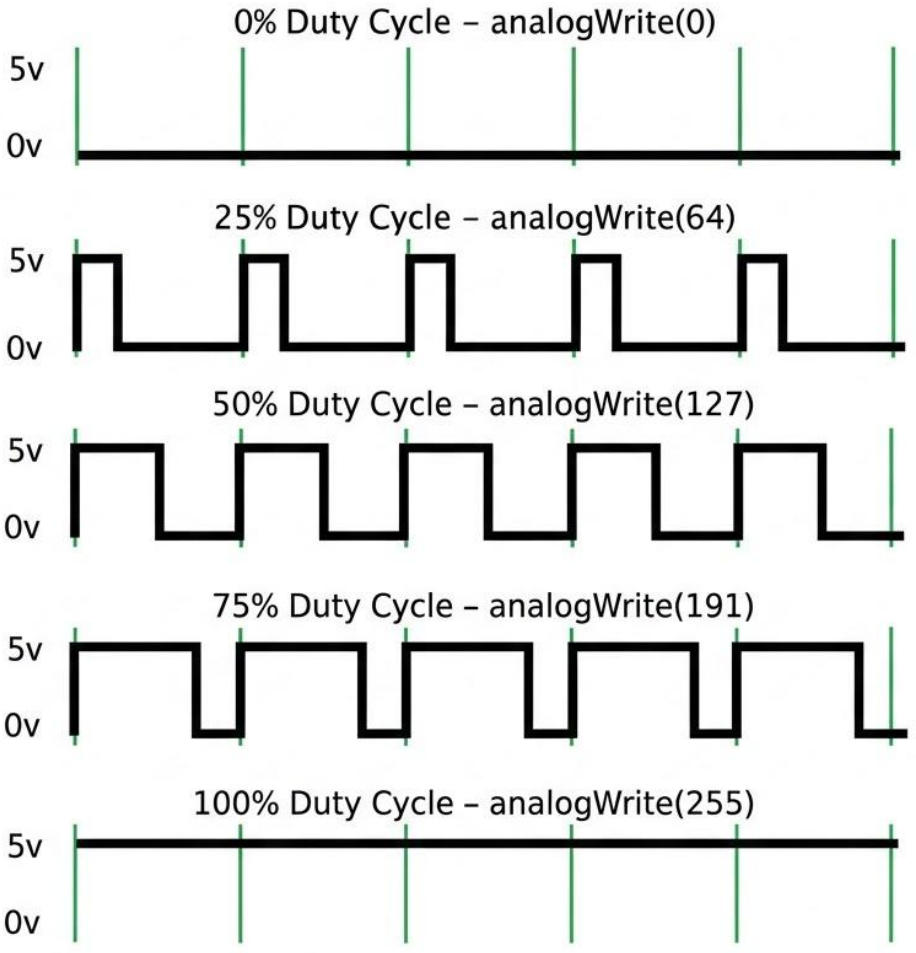
Duty Cycle Register

Defines the total period length of the generated output signal.

High-Time Register

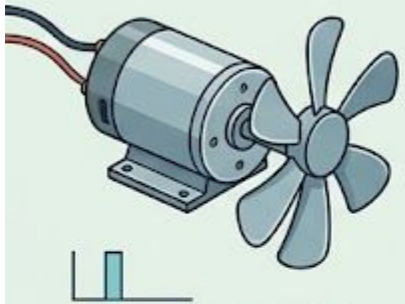
Specifies the duration within each period that the signal remains in a high state.




0% Duty Cycle – analogWrite(0)
5v
0v
25% Duty Cycle – analogWrite(64)
5v
0v
50% Duty Cycle – analogWrite(127)
5v
0v
75% Duty Cycle – analogWrite(191)
5v
0v
100% Duty Cycle – analogWrite(255)
5v
0v

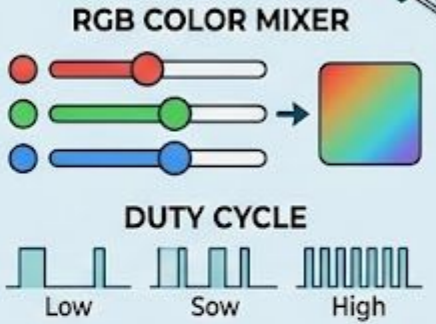
REAL-WORLD APPLICATIONS OF PULSE WIDTH MODULATION (PWM)

1. DC MOTOR SPEED CONTROL



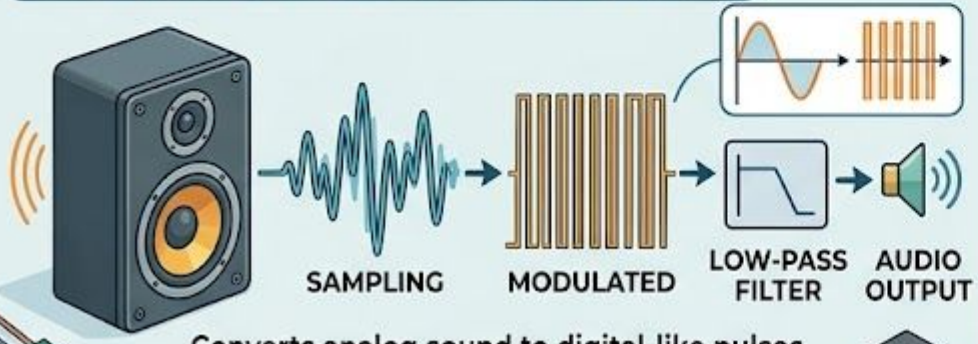
Low Duty Cycle → Slow

2. LED DIMMING & COLOR MIXING



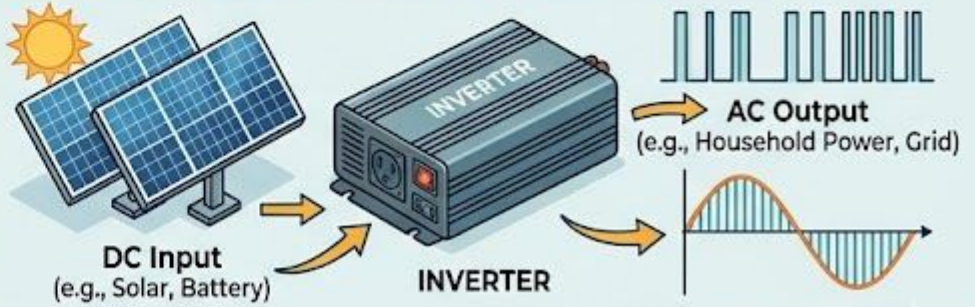
Controls brightness and enables trillions of colors

3. CLASS D AUDIO AMPLIFIERS



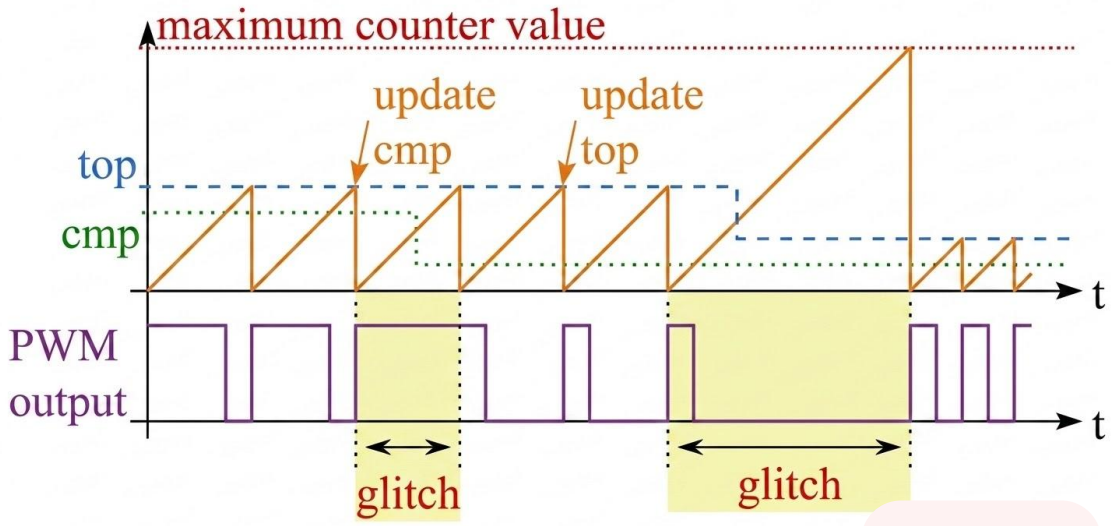
Converts analog sound to digital-like pulses for efficient, high-fidelity amplification

4. POWER INVERTERS (DC TO AC)



PWM efficiently recreates standard AC power from DC sources for homes and industry

PWM Signal Generation & Glitch Prevention



Standard Operation

- **HIGH**: Counter reaches zero
- **LOW**: Counter reaches **cmp**
- **RESET**: Counter reaches **top**

⚠ The Glitch Problem

Updating **cmp** or **top** values mid-cycle can cause timing inconsistencies, resulting in signal glitches.

To ensure a stable PWM signal, updates to control registers must be synchronized with the counter reset.

Watchdog Timer Fundamentals

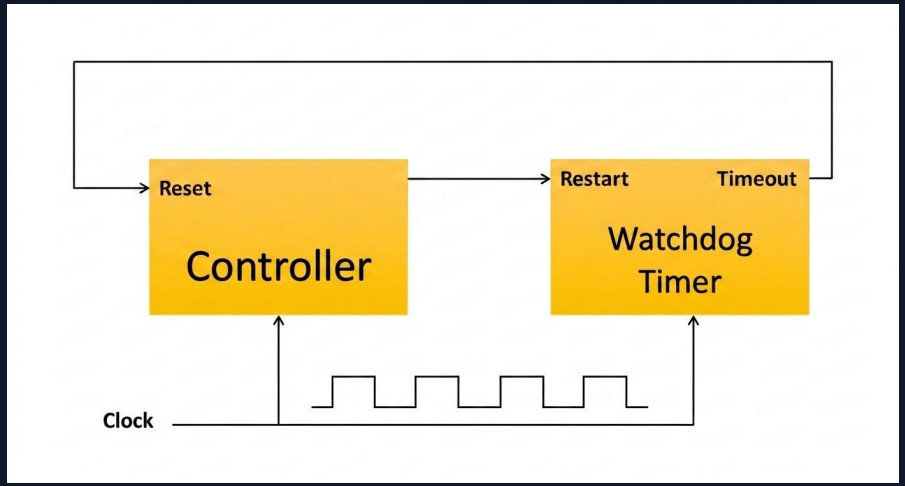
Primary Function: Used to monitor software execution and ensure system reliability.

How it Works

- Once enabled, it starts counting down automatically.
- Reaching **Zero** triggers a system reset.
- Reinitializes the controller and restarts the program.

Preventing Reset

- To avoid a controller reset, the software must reset the timer before it reaches zero.
- "Kicking the Dog": The periodic action of resetting the watchdog to prove the software is still running correctly.



Advanced Watchdog Features

Independent Oscillator

The Watchdog possesses its own dedicated internal oscillator, providing unique advantages:

- Immune to power-saving sleep modes
- Operates independently of the main system clock
- Ensures protection even when the MCU is dormant

Safe Configuration

Specific procedures are required to disable or modify Watchdog settings to prevent failure:

- Multi-step unlock sequences
- Prevents accidental deactivation by runaway code
- Locks settings after initial configuration

Robust hardware design ensures the Watchdog remains a reliable last line of defense against software failure.

Watchdog Timer Limitations

Target Applications

The watchdog verifies that specific program code positions are reached within a defined period.

If the program digresses from normal execution and fails to reset the watchdog, a system reset is triggered to resolve the issue.

⚠ Critical Limitation

The watchdog is ineffective if the program misbehaves but still manages to reset the timer, or if the underlying cause of failure persists even after a hardware reset.

Historical Case Study: NASA Mars Pathfinder (1997)

A famous example of a watchdog successfully identifying a program error but being unable to resolve the root cause due to the nature of the software failure.